

1. O que caracteriza uma API RESTful?

Uma **API RESTful** é uma interface de comunicação entre sistemas que segue os princípios da arquitetura **REST (Representational State Transfer)**. Ela utiliza requisições HTTP para permitir que aplicações troquem informações de forma padronizada.

As principais características de uma API RESTful são:

- utilização dos verbos HTTP (GET, POST, PUT e DELETE);
- URLs organizadas para representar recursos;
- comunicação sem estado (stateless);
- respostas geralmente no formato JSON;
- facilidade de integração entre diferentes aplicações.

Essas características tornam as APIs RESTful simples, escaláveis e amplamente utilizadas em aplicações web e mobile.

2. Quais são os verbos HTTP mais usados em APIs e suas funções?

Os principais verbos HTTP utilizados em APIs REST são:

Verbo	Função
GET	Consultar ou listar informações.
POST	Criar um novo registro.
PUT	Atualizar um registro existente.
DELETE	Excluir um registro.

Exemplo:

GET /usuarios

Lista os usuários.

POST /usuarios

Cria um novo usuário.

PUT /usuarios/5

Atualiza o usuário de ID 5.

DELETE /usuarios/5

Remove o usuário de ID 5.

3. Explique a vantagem de estruturar projetos em pastas como controllers, models e routes.

Organizar um projeto em camadas melhora a manutenção e facilita o desenvolvimento.

Cada pasta possui uma responsabilidade específica:

- **Routes:** definem os endpoints da API.
- **Controllers:** recebem as requisições, aplicam as regras de negócio e retornam as respostas.
- **Models:** realizam o acesso ao banco de dados.

Essa separação permite:

- código mais organizado;
- maior reutilização de funções;
- facilidade para manutenção;
- melhor escalabilidade;
- trabalho em equipe de forma mais organizada.

4. O que é Joi e para que serve?

Joi é uma biblioteca utilizada para validar dados recebidos pela aplicação antes que eles sejam processados ou armazenados no banco de dados.

Com ela é possível verificar se:

- campos obrigatórios foram preenchidos;
- e-mails possuem formato válido;
- senhas possuem tamanho mínimo;
- valores atendem às regras estabelecidas.

Exemplo:

```
const Joi = require("joi");
```

```
const schema = Joi.object({
```

```
  nome: Joi.string().min(3).required(),
```

```
  email: Joi.string().email().required(),
```

```
  senha: Joi.string().min(6).required()
```

```
});
```

5. Como a paginação melhora a performance de uma API?

A paginação divide os resultados de uma consulta em pequenas partes (páginas), evitando que todos os registros sejam retornados de uma única vez.

Suas principais vantagens são:

- reduz o consumo de memória;
- diminui o tráfego de dados;
- melhora o tempo de resposta;
- evita sobrecarga no servidor;
- facilita a navegação em grandes volumes de informações.

Exemplo:

GET /usuarios?page=2&limit=10

Retorna apenas os registros da segunda página, com dez resultados por página.

6. Qual é a finalidade do Swagger em projetos de API?

O **Swagger** é uma ferramenta utilizada para documentar APIs REST.

Ele permite:

- visualizar todos os endpoints disponíveis;
- consultar parâmetros aceitos;
- visualizar exemplos de respostas;
- testar requisições diretamente pelo navegador;
- facilitar a integração entre equipes.

A documentação gerada pelo Swagger torna a API mais organizada e profissional.

7. O que é o conceito de filtros em consultas?

Filtros permitem retornar apenas os registros que atendem a determinados critérios.

Por exemplo, em uma API de usuários, pode-se buscar apenas usuários cujo nome contenha uma determinada palavra.

Exemplo:

GET /usuarios?nome=Gabriel

Nesse caso, a consulta retornará apenas usuários cujo nome contenha "Gabriel".

Os filtros tornam as consultas mais eficientes e reduzem o volume de dados enviados ao cliente.

8. Explique como implementar um endpoint seguro para criar registros no MySQL.

Um endpoint seguro deve:

- validar os dados recebidos;
- utilizar consultas parametrizadas (Prepared Statements);
- tratar erros adequadamente;
- retornar códigos HTTP apropriados.

Exemplo:

```
const [resultado] = await pool.execute(
```

```
"INSERT INTO usuarios (nome, email) VALUES (?, ?)",
```

```
[nome, email]
```

```
);
```

Nesse exemplo, o uso de parâmetros (?) protege a aplicação contra ataques de SQL Injection.

9. Quais são os principais benefícios de validar dados na camada da API?

A validação garante que apenas dados corretos sejam processados pela aplicação.

Seus principais benefícios são:

- evita registros inválidos;
- reduz erros no banco de dados;
- aumenta a segurança da aplicação;
- melhora a experiência do usuário;
- protege contra entradas maliciosas.

Bibliotecas como **Joi** facilitam esse processo por meio da definição de esquemas de validação.

10. Como o Express simplifica o desenvolvimento de APIs RESTful?

O **Express** é um framework para Node.js que facilita a criação de servidores e APIs REST.

Ele simplifica o desenvolvimento por oferecer:

- criação simples de rotas;
- suporte a middlewares;
- integração com bancos de dados;
- tratamento de requisições HTTP;
- organização da aplicação em módulos.

Exemplo:

```
const express = require("express");
```

```
const app = express();
```

```
app.get("/usuarios", (req, res) => {
```

```
  res.json({
```

```
    mensagem: "Lista de usuários"
```

```
  });
```

```
});
```

Com poucos comandos é possível criar endpoints completos, tornando o desenvolvimento mais rápido e organizado.

